

ChatGPT - Mid

Mid-Semester Examination Syllabus and Guidelines

SWE 4401: Software Requirements and Specifications

June 28, 2025

Exam Syllabus

The mid-term examination will cover the following topics that we have discussed in class:

- Requirements Elicitation & User-Centric Artifacts:- Interpreting Stakeholder Needs- User Personas- User Stories- Acceptance Criteria (Gherkin Format)
- Requirement Quality & Analysis:- Characteristics of Good and Bad Requirements (e.g., Ambiguity, Testability, Epics)- Requirement Refinement Principles
- Agile Management & Prioritization:- The Requirements Hierarchy (Business, Functional, Non-Functional)- The MoSCoW Prioritization Method- Core Scrum Concepts (Roles and Events related to Backlog Management)

General Guidelines

- The exam questions will be designed to test your practical application of these topics.

Focus your preparation on understanding how these concepts connect and are used in a project scenario.

- A note on answering: Please be precise and concise. The questions are designed to be answered directly, and there is no need for long paragraphs to get full marks. If you are well-prepared, you should be able to complete the exam in approximately one hour.

Focus on quality and accuracy—less is more.

Generate a detailed not eabout these topics according to the guideline. Teach me everything available in the most understandable way possible

To tailor the most effective and understandable notes for you, could you please clarify the following:

1. What format do you prefer for the notes? (e.g., bullet points, structured outline, summary with examples)
2. Would you like real-world examples, diagrams, or Gherkin samples included?
3. Are there any specific areas among the listed topics you find most challenging and want more emphasis on?

Once I have that, I'll generate a complete, clear, and study-ready version of your mid-term syllabus notes.

1. points, and structured. summary at the end is also appreciated
2. yes
- 3.no. emphasis on all of them

Great. I'll create structured, point-based notes with clear explanations, real-world examples, diagrams or Gherkin samples where applicable, and a summary at the end. I'll cover all the topics listed in your SWE 4401 mid-term syllabus evenly and focus on practical understanding.

I'll let you know once the comprehensive study guide is ready.

 SWE 4401 Mid-Semester Exam Notes



Mid-Semester Exam Study Notes: Software Requirements and Specifications

Requirements Elicitation & User-Centric Artifacts

Interpreting Stakeholder Needs

- **Understanding Stakeholder Needs:** Requirements elicitation involves actively gathering input from stakeholders and *interpreting* their needs and expectations for the system . This means looking beyond exactly what stakeholders say to understand the *underlying goals or problems* they want to address.
- **Explicit vs. Implicit Needs:** Stakeholders may state some requirements explicitly, but others might be implicit or assumed. A good analyst listens for hints or assumptions. For example, a stakeholder might explicitly demand a feature (e.g. a housing project must reserve 30% parkland) but implicitly expect regular progress updates – failing to provide those updates could lose their support . Recognizing these unspoken needs is key to interpreting stakeholder requests correctly.
- **Elicitation Techniques:** Common techniques to uncover and clarify needs include interviews, workshops, surveys, observation, and prototyping. During these activities, it's vital to ask open-ended and probing questions to get to *why* a feature is needed, not just *what* is requested. This helps in distinguishing *wants* from true *needs*.
- **Clarification and Confirmation:** After gathering requirements, analysts should play back their understanding to stakeholders (through summaries, models, or user story descriptions) to confirm that the interpreted needs match what the stakeholder intended. This avoids misunderstandings early on. In requirements engineering, validation with stakeholders ensures you captured the right needs before moving forward.

User Personas

- **Definition:** *User personas* are **fictional characters** representing different user types of the system, created based on user research and real data . Each persona typically includes a name, a photo or avatar, and a narrative describing the person's demographics, goals, motivations, and pain points in context of the product.
- **Purpose:** Personas help the team empathize with users and keep a **user-centric focus** in development. They answer the question "Who are we building this for?" By personifying user groups, the team can discuss features in terms of how *that specific persona* would use them or benefit . This prevents designing for "the average user" and instead keeps real-world perspectives in mind.

- **Usage in Projects:** Agile teams sometimes use personas to guide decisions about product features and priorities. For example, a persona “Alice the Busy Executive” might value time-saving features, whereas “Bob the Novice User” might need simplicity. These guides inform which requirements are more important for each user type. In Scrum, personas are an optional tool, but they can support writing better user stories and focusing on delivering value to specific user groups .
- **Persona Details:** A good persona description includes the user’s background (role, experience, environment), their main objectives or tasks they want to accomplish with the product, and their frustrations or needs. **Example:** “Jane Doe, 34, is a tech-savvy marketing manager who needs to quickly generate campaign reports. She values efficiency and intuitive interfaces, as she doesn’t have time for training.” Such details help the team **avoid personal biases** and design for the persona’s perspective rather than the developers’ preferences .

User Stories

- **Definition:** A *user story* is an informal, brief description of a software feature or requirement told from an **end user’s perspective** . It is essentially a one-sentence statement of something the user needs or wants to do with the system, focusing on the value delivered to the user. User stories are fundamental in agile requirement management, replacing traditional detailed requirements with **conversations and confirmation**.
- **Format:** A common format for user stories is: “As a **<role>**, I want **<goal/feature>** so that **<benefit>**.” This template captures **who** wants something, **what** they want, and **why** they want it . For example: “As an **online banking customer**, I want to **view my account balance** so that **I can track my spending easily**.” This format ensures each story is tied to a specific user role and a clear benefit or rationale.
- **Characteristics:** A good user story has certain quality attributes often summarized by the acronym **INVEST**:
 - *Independent:* It stands alone (scheduling one story doesn’t inherently depend on another).
 - *Negotiable:* It’s not a detailed contract; details can be discussed and changed.
 - *Valuable:* It delivers clear value to the end user or customer.
 - *Estimable:* The team can roughly size it (understand the effort to implement).
 - *Small:* It can be completed in a single iteration (not too large/complex).

- **Testable:** It has clear acceptance criteria so it can be verified when done
- **Epics and Hierarchy:** Large user stories often called **Epics** (e.g., “Implement online shopping cart”) need to be broken down into smaller user stories that fit the INVEST criteria. Epics serve as placeholders for broad features and are refined into multiple detailed stories over time. Multiple related user stories can also group into *themes* or *features*, but the user story remains the **smallest unit of work** in agile planning
- **Communication Tool:** The true power of user stories is to facilitate discussion. Rather than exhaustive documentation upfront, a story’s brevity invites the team and stakeholders to discuss details during backlog refinement and implementation. In fact, a user story is sometimes described as “a promise for a conversation.” The details (known as acceptance criteria or conditions of satisfaction) are fleshed out as the team talks through how to satisfy the story’s intent.

Acceptance Criteria (Gherkin Format)

- **Definition:** *Acceptance criteria* are the **conditions or requirements** that a product must meet for the user story to be considered complete and acceptable to the stakeholder. They specify the **boundaries and expected outcomes** for the functionality described in the user story. In other words, acceptance criteria answer the question: “How will we know when this story is done *correctly*?”.
- **Format and BDD/Gherkin:** A common way to write acceptance criteria is using the **Gherkin syntax** (from Behavior-Driven Development). Gherkin uses a “**Given – When – Then**” structure to define scenarios:
 - **Given** some initial context (preconditions),
 - **When** an action or event occurs,
 - **Then** observe the expected outcome (postcondition).

This format is very readable for both business stakeholders and developers. It allows acceptance criteria to double as **acceptance test cases**. For example, for a login feature:
Scenario: Successful login

Given a registered user with valid credentials, **When** the user attempts to log in with correct username and password, **Then** the user is granted access to the system (and sees their dashboard).

- **Clarity and Testability:** Writing criteria in Gherkin forces clarity. Each scenario describes a concrete example, which makes the requirement **testable**. Business stakeholders can read it to verify it captures the intent, and developers/testers can use it to implement automated tests. In fact, the Gherkin language is specifically designed to be **unambiguous and executable** by tools like Cucumber for automated acceptance testing.
 - **Multiple Scenarios:** Often, a single user story will have multiple acceptance criteria or scenarios covering different aspects or edge cases. For instance, besides the successful login scenario, you'd have: *"Scenario: Login fails with wrong password"* etc. Each scenario follows the Given/When/Then format to specify expected behavior in that case.
 - **Example (Gherkin Acceptance Criteria):**
Feature: Account Balance Display
Scenario: Show balance after successful login
Given I have logged into the mobile banking app,
When I navigate to the Accounts page,
Then I should see the balance of each of my accounts on screen.
Scenario: Do not show balance if not logged in
Given I am **not** logged into the mobile app,
When I open the Accounts page,
Then I should see a login prompt **and not** see any account balances.
- These criteria set clear conditions that developers must meet and testers can validate.

Requirement Quality & Analysis

Characteristics of Good vs. Bad Requirements

- **Clarity and Lack of Ambiguity:** A *good requirement* is **unambiguous** – it is stated in clear, precise language that cannot be easily misinterpreted. Each reader (stakeholder, developer, tester) should interpret it the same way. A *bad requirement* is often vague or ambiguous, leaving room for multiple interpretations. For example, *"The system shall respond quickly to user queries"* is ambiguous – what does *"quickly"* mean (1 second? 1 minute?) . A clearer requirement would specify a measurable response time (e.g., *"within 2 seconds"*).

- **Testability:** Good requirements are **testable (verifiable)** – you can design a test to check if the requirement is met . This usually means the requirement is *specific and measurable*. Non-testable requirements use subjective terms like “*user-friendly*” or “*efficient*” without criteria, or include placeholders like “TBD”. Such terms make a requirement bad because different people may judge them differently, and you can’t objectively verify them . Always ask: “How would I verify this requirement is fulfilled?” If no clear answer, the requirement likely needs refinement.
- **Complete and Atomic:** A well-written requirement expresses **one discrete idea** (often called *atomic*) and is complete in itself. Bad requirements might bundle multiple ideas together or leave important details out. For instance, “*The system shall display the user’s profile and allow editing and send notifications*” is combining multiple functions; splitting into separate requirements (display profile, edit profile, send notifications) makes them atomic. Completeness means the requirement provides all necessary information – if additional assumptions are needed to understand it, it may be incomplete.
- **Feasible and Necessary:** Good requirements are **feasible** (realistic to implement within constraints of time, budget, technology) and **necessary** for the system’s purpose . If a requirement cannot be implemented with known technology or is out-of-scope, it’s not a good requirement. Also, each requirement should tie back to a real stakeholder need or business objective (necessity). A bad requirement might describe a feature that sounded good but isn’t actually needed or cannot be delivered given project constraints.
- **Consistent and Non-Conflicting:** Each requirement should not conflict with any other requirement or higher-level specification. Inconsistencies (one requirement says “do X” and another says “don’t do X”) are a sign of poor requirement quality.
- **Example – Ambiguity Fix:** Consider the requirement “*The system shall not accept passwords longer than 15 characters.*” At face value it seems clear, but it raised ambiguity: does “not accept” mean the user can’t input more than 15 chars, or the system will truncate extra characters, or it will show an error? This was clarified by adding specificity: “*If the user enters more than 15 characters for a password, the system shall display an error message and refuse the input.*” Now the requirement is explicit about the expected behavior .

- **Epics vs. Refined Requirements:** An *Epic* (very broad, high-level requirement or user story) is generally **too coarse-grained** to be a “good requirement” on its own. Epics are considered *bad requirements* until they are **refined** into more specific, actionable ones. For example, “*Implement inventory management module*” is an epic – it needs to be decomposed into many smaller, clear requirements or user stories (such as “scan product barcode to add an item to inventory” or “manager can view inventory reports”) to be implementable and testable. Leaving requirements as big epics leads to ambiguity and lack of testability. Good practice is to break down epics into smaller **functional requirements or user stories** that meet the qualities above.

Requirement Refinement Principles

- **Progressive Elaboration:** Refinement is the process of starting with high-level ideas and **gradually elaborating** them to add detail and clarity over time. Early in a project, requirements might be broad (e.g., epics or general stakeholder needs). Through discussions, prototyping, and feedback, these are refined into detailed requirements and specifications. The idea is not to get everything perfectly detailed upfront, but to iterate – revisit and flesh out requirements continuously as understanding improves.
- **Splitting and Decomposing:** A key refinement principle is **divide and conquer**. Large requirements should be broken into smaller ones that are easier to understand and implement. For functional requirements in agile, this means breaking Epics into user stories (and stories into tasks if needed). Techniques for splitting user stories include by workflow steps, business rules, data types, user personas, etc. The goal is that each resulting item is **focused and clear**. Splitting also helps in prioritization – smaller items can be independently scheduled and delivered.
- **Just-In-Time Detail:** In agile environments, teams often wait to refine requirements until **just before** they will be developed (e.g., refining the next sprint’s stories a sprint or two in advance). This ensures you have the *latest information* and avoid wasting effort detailing something that might change. It aligns with the idea that requirements are “**emergent**” – they evolve as we learn more. However, you must ensure that by the time a team commits to a story, it is sufficiently refined (clear, estimated, and testable). The Scrum Guide notes that **Product Backlog refinement is an ongoing activity** throughout the sprint to add detail and break down items so that they are ready for future sprints

- **INVEST and Quality Criteria in Refinement:** During refinement, teams apply quality checklists (like the characteristics of good requirements) to each requirement. For user stories, the **INVEST** criteria discussed above serves as a guideline – if a story isn't independently implementable or testable, continue refining. For traditional requirements, teams might use a checklist: *Is it clear? Feasible? Testable?* If not, rewrite or split further.
- **Stakeholder Feedback Loop:** Good refinement involves verifying the updated understanding with stakeholders. For example, after refining an epic into several user stories with acceptance criteria, a team might review them with the Product Owner or client to ensure they still meet the stakeholder's intent. This prevents the "telephone game" effect where the meaning drifts as teams rewrite requirements.
- **Continuous Process:** Refinement isn't a one-time phase; it's **continuous**. Requirements can be revisited and refined even later in the project if new information comes to light or if the scope changes. Agile teams often allocate a portion of each sprint (e.g., ~10% of time) for backlog refinement activities . This could involve the Product Owner and development team grooming the backlog: reprioritizing items, clarifying acceptance criteria, and estimating effort. The continuous nature ensures the backlog (the set of all requirements/stories) remains current and well-understood.

Agile Management & Prioritization

Requirements Hierarchy: Business, Functional, Non-Functional

In requirements engineering, requirements exist at **different levels**, and understanding their hierarchy helps in analysis and management:

- **Business Requirements:** These are high-level goals, objectives, or needs of the organization or project. They answer "*Why are we doing this project?*" and "*What value or outcomes do we expect?*" Business requirements typically come from business stakeholders or project sponsors and describe the desired business impact (e.g., increase market share by 5%, comply with new regulation, improve customer satisfaction). They are usually solution-agnostic (not describing specific features, but the problem or opportunity). **Example:** "*The system shall enable a 20% reduction in processing time for customer orders.*"

- **Functional Requirements:** These specify **what the system should do** – the features, behaviors, and functions of the software. They detail the **functional capabilities** to be implemented to fulfill the business requirements and user needs . Each functional requirement defines an input, behavior, or output of the system. They can be documented in various forms (traditional “The system shall...” statements, or as user stories/use cases in agile). **Example:** *“The system shall send an email confirmation to the user after an order is placed.”* Functional requirements collectively describe the *functionality* of the product.
- **Non-Functional Requirements (NFRs):** These describe the **quality attributes, constraints, and other characteristics** the system must have, other than specific behaviors . They are sometimes called “quality requirements” or “constraints”. Non-functional requirements often answer *“How well or under what conditions does the system perform its functions?”* They include:
 - **Performance:** e.g., response time, throughput, scalability.
 - **Security:** e.g., authentication, authorization, data protection measures.
 - **Usability:** e.g., ease of use, accessibility standards.
 - **Reliability:** e.g., uptime, fault tolerance.
 - **Maintainability:** e.g., modularity, ease of fixing bugs or making changes.
 - **Compliance:** e.g., standards or regulations the system must adhere to.

Even though labeled “non-functional,” these requirements are critical to system success and often have specific metrics or targets (e.g., *“99.5% uptime”, “page load under 2 seconds for 95% of requests”*).

- **Interrelationships:** Business requirements set the direction and success criteria for the project. Functional and non-functional requirements (*solution requirements*) are then derived to satisfy those business needs . For instance, a business requirement to *improve customer retention* might lead to functional requirements for a loyalty program feature, and non-functional requirements for system availability (since downtime could frustrate users). Ensuring traceability between these levels (each functional/NFR can be traced back to a business goal) helps maintain scope and prioritize what matters.

- **Example Hierarchy:** *"Increase online sales by 15% (Business Req). To achieve that, the system needs an online storefront (Business/Sub-Business Req). The storefront must allow users to browse products and place orders (Functional Reqs: browse catalog, add to cart, checkout, etc.). The site should handle 10,000 concurrent users with sub-second page loads (Non-Functional: performance requirement)."* This illustrates how lower-level requirements support higher-level goals.

MoSCoW Prioritization Method

The **MoSCoW method** is a popular technique for prioritizing requirements by assigning them to four categories of importance :

- **M – Must Have:** These are requirements that are absolutely **essential** for the system's success or for a viable product. *Must-haves* represent the Minimum Usable Subset of requirements – if even one "Must" is not delivered, the project (or release) would be considered a failure . They have highest priority and are often non-negotiable (e.g., core features, legal compliance items).
- **S – Should Have:** Requirements that are **important** and highly desired, but not strictly mandatory for the first delivery. *Should-haves* add significant value or usability, but the project can still succeed without them in the short term . If necessary, should-haves can be deferred to a later release, but product owners will aim to include as many shoulds as possible after all musts are satisfied.
- **C – Could Have:** Requirements that are **desirable but not necessary**. These are often enhancements or nice-to-have features that improve user experience or satisfaction with relatively low effort or cost . They are the first candidates to drop if the schedule or resources are tight. Including them is a bonus if time permits, but leaving them out won't break the core functionality.
- **W – Won't Have (this time):** Requirements that have been agreed upon by stakeholders as the **lowest priority** or out of scope for the current timeframe . *Won't-haves* (sometimes phrased as "Would like to have but not now") will not be delivered in the upcoming release or iteration. This category sets clear expectations that these items are *not* being worked on at present, which helps avoid scope creep. They may be revisited in the future.

- **Usage:** The MoSCoW method is typically applied in collaboration with stakeholders (e.g., during requirements workshops or backlog grooming sessions) to achieve a shared understanding of what's critical versus optional . It's especially useful in time-constrained projects (like timeboxed agile projects) – where you fix time/scope and prioritize within that. Teams focus on delivering all **Musts** first, then **Shoulds**, and so on. If delays occur, **Shoulds** and **Could**s are the ones to cut or postpone, ensuring **Musts** are done.
- **Example:** Suppose we're building a mobile app for a bank:
 - *Must:* Users **must be able to log in** securely and view their account balance (without these, the app fails its basic purpose) – **Must Have**.
 - *Should:* Users *should be able to deposit checks via photo capture* (important feature, but if it slips to next release, core banking still works) – **Should Have**.
 - *Could:* The app *could include custom theming options* for user interface (nice to have, not crucial to banking functionality) – **Could Have**.
 - *Won't:* Integration with third-party budgeting apps is decided to be out of scope for now – **Won't Have (this time)**.

Using MoSCoW, the team and stakeholders clearly know what to focus on first and what to defer.

Core Scrum Concepts (Roles and Events for Backlog Management)

Scrum is an agile framework that uses specific roles, events, and artifacts to manage work. For effective **backlog management** (handling requirements in Scrum), it's important to understand the roles responsible and the events where backlog items are discussed and updated:

- **Product Backlog (Artifact):** *Before detailing roles/events, note that the **Product Backlog** is the central artifact in Scrum for requirements. It is an **ordered list** of everything known to be needed in the product – essentially the team's to-do list of user stories, features, and bug fixes. It is owned and managed by the Product Owner, but it's visible to all and anyone can suggest items. The backlog is **dynamic**; it evolves as the product and environment evolve. Crucially, the backlog must be transparent, prioritized, and regularly refined so that the most important items are ready for implementation*
- **Scrum Roles in Backlog Management:**

- Product Owner (PO):** The PO is *accountable for the Product Backlog*. They represent the voice of the customer/stakeholders and ensure the team is working on the highest-value items. **Key backlog responsibilities of the PO include:** defining and expressing product backlog items clearly, ordering the items by priority/value, and ensuring the backlog is visible and understood . The PO decides what gets built and in what order, balancing the needs of the business and users. They continuously collaborate with stakeholders to gather requirements and with the team to refine those items. In practice, the PO adds new items, updates acceptance criteria, and removes items that are no longer needed. No one can force the team to work on something not in the backlog or change the order without involving the PO – this single authority maintains clarity and focus .
- Development Team (Developers):** The developers are the ones who implement the backlog items. They **pull work** from the backlog (top-priority items) during Sprint Planning to execute in a Sprint. While the PO defines *what* needs doing, the team decides *how* to do it. The dev team contributes to backlog refinement by estimating items and clarifying details with the PO. They are encouraged to be proactive in suggesting new backlog items (like technical improvements) or flagging items that need more detail. **During a Sprint**, the selected Product Backlog items become the **Sprint Backlog** – the list of items the dev team commits to deliver in that iteration . The developers own the Sprint Backlog and adjust it as needed during the Sprint (if some tasks are taking longer, etc.), in coordination with the PO for scope changes. The dev team's ability to self-organize is key: they decide how to slice the work, which team member does which item, and so on, to meet the Sprint Goal.
- Scrum Master:** The Scrum Master facilitates the Scrum process and helps remove impediments. In terms of backlog, the Scrum Master **coaches the PO and team on effective backlog management practices** . They help the Product Owner in techniques for ordering and refining the backlog, and ensure the team understands the importance of clear and actionable backlog items. The Scrum Master may facilitate backlog refinement meetings (also called grooming sessions) to make sure they are efficient and that the team isn't overwhelmed or skipping refinement. They also ensure that Scrum events (like planning and review) occur and are time-boxed, which indirectly keeps backlog management on track . If organizational issues (e.g., too many last-minute changes or trouble getting stakeholder feedback) hinder backlog quality, Scrum Masters work to resolve those.

- **Scrum Events related to Backlog:**

- **Product Backlog Refinement:** Although the Scrum Guide treats refinement as an ongoing activity rather than a formal event, teams often hold regular **backlog refinement sessions** (mid-sprint or as needed). In these sessions, the PO and dev team review upcoming backlog items to clarify requirements, split stories or redefine them if too large, and assign preliminary estimates. The goal is to keep the backlog healthy – top items are **“ready”** (clear, estimated, testable) for the next Sprint or two . A common guideline is to spend up to ~10% of the team’s capacity on refinement . By refining continuously, the team can go into Sprint Planning already understanding the highest priorities.
- **Sprint Planning:** This event kicks off the Sprint. Here, the Product Owner presents the **highest priority backlog items** to the team, and the team decides which ones to include in the Sprint. Sprint Planning has two main parts: **What** will be done, and **How** it will be done. In the “what” part, the PO and team agree on a Sprint Goal and select relevant Product Backlog items that achieve that goal. The PO must come prepared with an ordered backlog and ideally that the top items are well-refined . The team may do final refinements or split stories during planning if needed. Once the “what” is decided, the items move to the Sprint Backlog and the team plans “how” to deliver them (designing tasks, etc.). By the end, the team makes a commitment to the Sprint Backlog items + Sprint Goal.
- **Daily Scrum (Daily Stand-up):** This is a short daily team meeting. Team members sync up on progress **towards the Sprint Backlog items**. While not directly about the Product Backlog, it’s about managing the *Sprint Backlog*. They might adjust their plan for the day to stay on track. If a Sprint Backlog item is blocked or done, they might pull another from the backlog (with PO agreement) or collaborate to solve issues. The daily scrum ensures transparency on how backlog items in the sprint are moving.

- **Sprint Review:** This event happens at the end of the Sprint. The Scrum Team presents the **completed Increment** (finished backlog items) to stakeholders for feedback. Crucially, the Sprint Review is also a time to **adapt the Product Backlog** based on this feedback . Stakeholders might request new features or changes, which the Product Owner will add or re-prioritize in the backlog. The team and PO review what went well and what is coming next: *“Given what we’ve completed and learned, what are the most important things to do next?”* The backlog is thus updated – new items may be added, done items marked completed, and priorities adjusted.
- **Sprint Retrospective:** This is about the process (how the team works) rather than the product content. Indirectly, a retrospective can impact backlog management if the team decides on improvements like “we need to refine requirements more before Sprint Planning” or “let’s involve the UX designer earlier on backlog items.” These process changes can improve how well the backlog is maintained.
- **The Sprint (as a container):** The Sprint itself is the timebox during which all above events and development happen. During the Sprint, as per Scrum rules: no one should change the Sprint Backlog arbitrarily; scope can only be negotiated with the PO if needed . However, the *Product Backlog* (future items) can still be refined during the Sprint for future readiness. By Sprint’s end, the team should have potentially shippable product increments that meet the acceptance criteria of those backlog items.

In summary, Scrum’s approach to requirements is *highly collaborative and iterative*. The Product Backlog is constantly managed by the PO with input from the team, ensuring that at any given time, everyone knows what the priorities are and what the details of those top items entail. The roles each have a part to play: the PO drives *what* gets built, the team decides *how/when* within sprint cycles, and the Scrum Master ensures the process runs smoothly. Regular events (planning, review, refinement sessions) provide structured touchpoints to keep the requirements (backlog) aligned with stakeholder needs and project goals.

Summary

- **Requirements Elicitation & User-Centric Artifacts:** Focuses on gathering the true needs of stakeholders and users. Key practices include interpreting stakeholder input (reading *implicit* needs beyond what is said), and using **user-centric artifacts** like *Personas* (fictional user archetypes for empathy), *User Stories* (short, user-focused requirement statements in everyday language), and *Acceptance Criteria* (specific conditions in Given-When-Then format to validate a story's completion). These techniques ensure requirements remain focused on user value and are clear and testable .
- **Requirement Quality & Analysis:** Emphasizes writing *good requirements* that are clear (no ambiguity), feasible, and testable. Good requirements are often **atomic** (one thing at a time), **unambiguous**, **measurable**, and tied to a real need. *Bad requirements* suffer from vagueness, subjectivity (e.g. "easy to use"), or over-breadth (e.g. big "epics" that need breakdown). Requirements should be continuously **refined** – large or unclear items are iteratively detailed and split into smaller ones until they meet quality criteria. This refinement process is ongoing, involving stakeholder feedback and team collaboration to flesh out details "just in time" for development.
- **Agile Management & Prioritization:** Introduces how to organize and prioritize requirements in an agile setting. The **requirements hierarchy** spans high-level *Business Requirements* (project objectives) down to *Functional* and *Non-Functional Requirements* (specific capabilities and quality attributes) . Prioritization methods like **MoSCoW** (Must/Should/Could/Won't) help teams focus on what's truly essential , ensuring critical requirements are delivered first. **Scrum** is highlighted as a framework where specific roles (*Product Owner*, *Scrum Master*, *Development Team*) and events (*Refinement*, *Sprint Planning*, *Daily Scrums*, *Sprint Review*, etc.) center around managing the **Product Backlog** – the live collection of requirements. The Product Owner drives backlog priorities, the team builds the backlog items into increments, and Scrum events provide a rhythm for continuous feedback and backlog adjustment. This framework of roles and ceremonies keeps requirement management adaptive and aligned with stakeholder needs throughout the project.

By understanding and applying all these concepts – from eliciting stakeholder needs with user stories and acceptance tests, to writing high-quality requirements, to agile prioritization and continuous backlog refinement – you will be well-prepared to handle practical scenario-based questions in the exam. Focus on the **connections**: for example, how a well-written user story (with good criteria) flows into the backlog, how the Product Owner's prioritization using MoSCoW or business value determines what the team works on, and how the team's understanding of good requirement qualities ensures the delivered software meets the intended need. In essence, the **end-to-end lifecycle** of a requirement (from elicitation to implementation in Scrum) is covered by these topics – mastering them will enable you to develop software that truly satisfies user needs, with clarity and agility.



Sources

